说明

CH32X035、CH32H417、CH32V205 等系列 MCU 产品配备了一个称为 PIOC 的外设。本文以 CH32X035 及配套 1_Wire 工程为例，介绍 PIOC 的执行结构、主机数据交互和时序设计，并通过 WS2812 单线驱灯说明程序装载、汇编取数、逐位发送及结果处理的方法。

适用范围

适用范围	系列
通用 MCU	CH32X035、CH643、CH32V205、CH32H41x 等

目录

说明 1
目录 2
表格索引 2
图片索引 2
第 1 章 PIOC 功能结构 1
1.1 结构总览 1
1.2 主机与 PIOC 交互 2
1.3 时序设计 3
第 2 章 WS2812 单线驱灯示例 6
2.1 协议时序窗口 6
2.2 工程组成及初始化 6
2.3 命令字与结果码 6
2.4 短帧与长帧路径 7
2.5 主机调用与一次完整发送 9
历史版本 10
声明 11

表格索引

PIOC 主要特性 1
示例工程主要文件 6
例程的命令字分配 7
主机两种发送接口的操作顺序 9

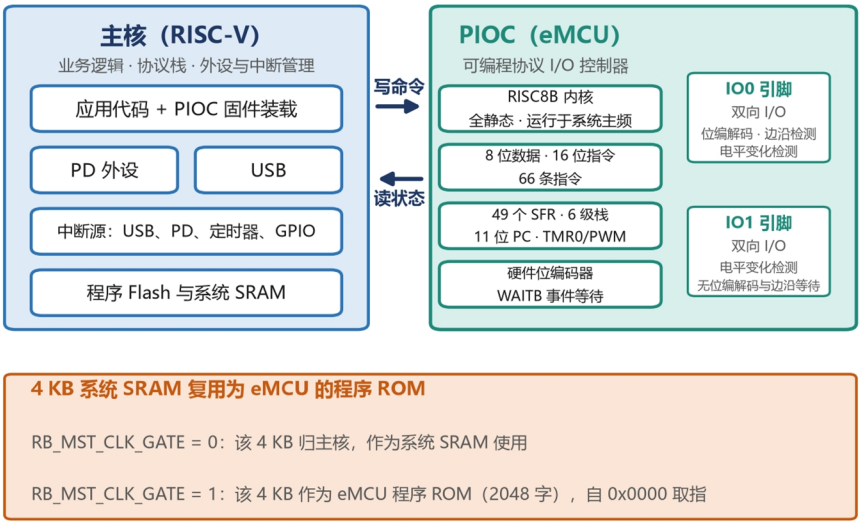
图片索引

CH32X035 PIOC 结构 1
命令与结果的三个状态位 2
两种数据传递方式 2
软件分配两块 SFR 缓冲 3
指令占用与边沿间隔 4
PIOC 两种位调制 4
定时器模式的计数 5
PWM 模式的计数 5
WS2812 位定义 6
代码中三条路径 7
软件位时序 8

第1章 PIOC 功能结构

1.1 结构总览

图 1-1 CH32X035 PIOC 结构



PIOC 是芯片内部集成的可编程协议 I/O 控制器，手册中亦称 eMCU。其内部为 RISC8B 内核，拥有独立的程序空间、寄存器组与指令流，可在不占用主机中断与调度资源的前提下，按确定的指令周期在两根引脚上产生协议波形。

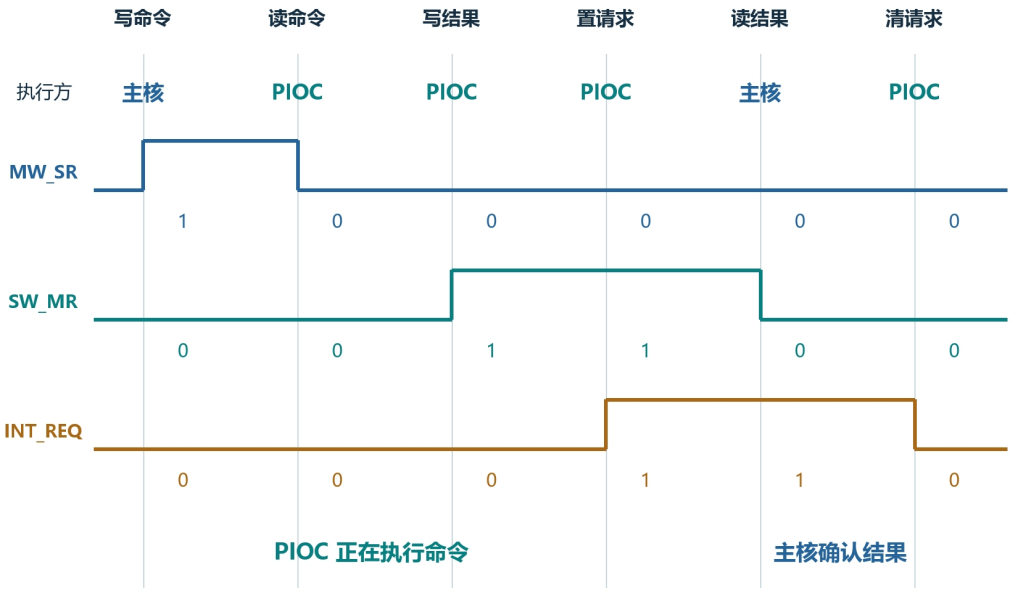
从使用方式看，主机与 PIOC 之间的交互集中在三个环节：装载（把编译好的指令写入共享 SRAM 并交由 PIOC 取指）、下发任务（写入命令与参数）、读取结果（事务结束后读取结果码）。完成装载后，协议执行由 PIOC 独立完成，主机可继续执行其它任务。因此，PIOC 的定位可以概括为：一颗可由用户编程的 8 位协议控制器，用于实现时序确定、速率中等、引脚数量少的接口。其功能由用户编写的汇编程序定义，而非由芯片固定。

表 1-1 PIOC 主要特性

特性	规格
内核	RISC8B，8 位数据、16 位指令、66 条指令。
运行频率	与系统主频相同，全静态设计，时序按周期计算。
程序空间	2048 字（4KB），由系统 SRAM 复用
寄存器	49 个 SFR（0x00 – 0x3F），含 33 字节数据寄存器
堆栈	6 级，11 位宽
引脚	2 路双向 I/O，支持电平变化检测
位编解码	PWM（25%/75%）与 Manchester，位宽可编程
定时器	8 位主动重加载定时器 T0，定时器/PWM 两种模式
装载方式	支持暂停与动态加载

1.2 主机与 PIOC 交互

图 1-2 命令与结果的三个状态位



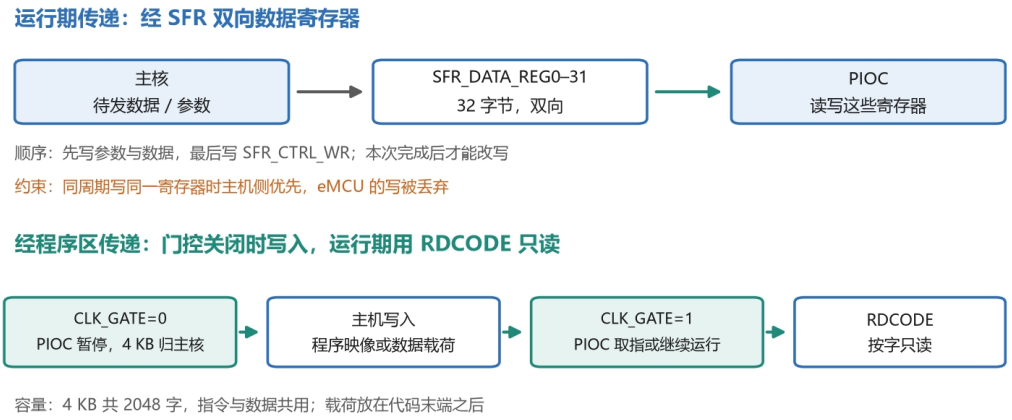
MW_SR 和 SW_MR 随寄存器读写自动变化；本图 INT_REQ 由示例固件控制

MW_SR 表示命令是否已被 PIOC 取走（主机写入 CTRL_WR 后 MW_SR 自动置 1，PIOC 读取 CTRL_WR 后 MW_SR 自动清 0，只表示命令已被取走，此时协议处理可能刚开始。只有结合本次提交动作，才能解释这个标志，不能把初始值 0 当作完成）；

SW_MR 表示结果是否已被主机读取（PIOC 写入 CTRL_RD 后 SW_MR 自动置 1，主机读取 CTRL_RD 后 DATA_SW_MR 自动清 0）；

INT_REQ 由 PIOC 置位，用于向主机请求中断或取消请求，主机读走结果后由固件清 INT_REQ。主机可以通过对只读寄存器 CTRL_RD 的假写操作实现对 INT_REQ 清 0，主机写 CTRL_RD 对其中数据无实际影响。

图 1-3 两种数据传递方式



SFR 方式适合参数和少量数据。双方先约定寄存器用途，例如 DATA0 由主机提交前写入，DATA1 由 PIOC 执行时写入，主机收到完成通知后读取。主机与 PIOC 同周期写同一共享寄存器时，主机写优先，PIOC 写被丢弃。

程序区方式适合较多的只读数据或常量表。该 4KB 在 CLK_GATE=0 时归主机、CLK_GATE=1 时作为 PIOC 程序 ROM，因此写入必须在门控关闭（或保持复位）期间完成；写完再把 CLK_GATE 置 1。PIOC 读取时使用 RDCODE：把目标地址低 8 位写入 SFR_INDIR_ADDR、高 3 位写入 A 寄存器，一次取回 16 位数据，地址按 11 位字地址计算。程序区共 2048 字，指令与数据共用；PIOC 只能读、不能写，它产

生的结果只能经 SFR 交回主机；CLK_GATE=1 期间主机也不得写这块区域，需要更新数据时必须重新关闭门控。

图 1-4 软件分配两块 SFR 缓冲



单次数据放不下时，可以把可用 DATA_REG 分成两块。PIOC 读取 A 时，主机只写 B；交换前必须同时满足“A 已读完”和“B 已准备好”。接收方向反过来：PIOC 写入空块，主机读取已完成块。块长度、有效标志、最后一块长度和超时处理都需要软件定义。EVT 示例中 UART、IIC 等例程包含分段数据通知，可参考其寄存器分配与计数方式。

1.3 时序设计

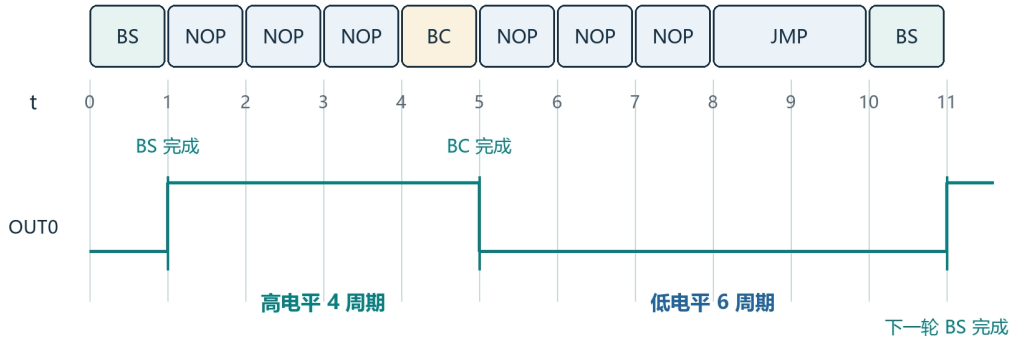
PIOC基于单时钟周期的精简指令集RISC8B内核，运行于系统主频，时序设计以指令周期为单位。协议手册给出的通常是纳秒或微秒，需先换算为周期数。PIOC没有专用延时指令，延时由执行空操作构成，按长度可分为三档：1～10周期可采用连续NOP；10～100周期可调用一个含n条NOP的子程序（周期数为n+4，CALL与RET各占2个周期）；长周期可采用计数循环，每轮DEC与JNZ合计3个周期。

把一段代码换算成波形，只需沿执行路径累加周期。下例假设IO0已配置为软件输出，位编码与定时器输出均已关闭：

```
PULSE:  BS    SFR_PORT_IO,SB_PORT_OUT0
        NOP
        NOP
        NOP
        BC    SFR_PORT_IO,SB_PORT_OUT0
        NOP
        NOP
        NOP
        JMP   PULSE
```

图中按指令完成边界表示输出锁存值的变化：第一条BS完成于t=1，BC完成于t=5，下一轮BS完成于t=11。高电平为4周期，低电平为6周期，48MHz下每周期约20.833ns，名义高、低电平分别约83.333ns和125ns。普通传送、运算和位操作通常为1周期；JMP、CALL和返回为2周期；条件转移成立或发生跳过时计入额外周期；RDCODE为3周期。WAITB的总时间由事件何时成立决定，不能把等待时间作为固定的单周期延时。

图 1-5 指令占用与边沿间隔

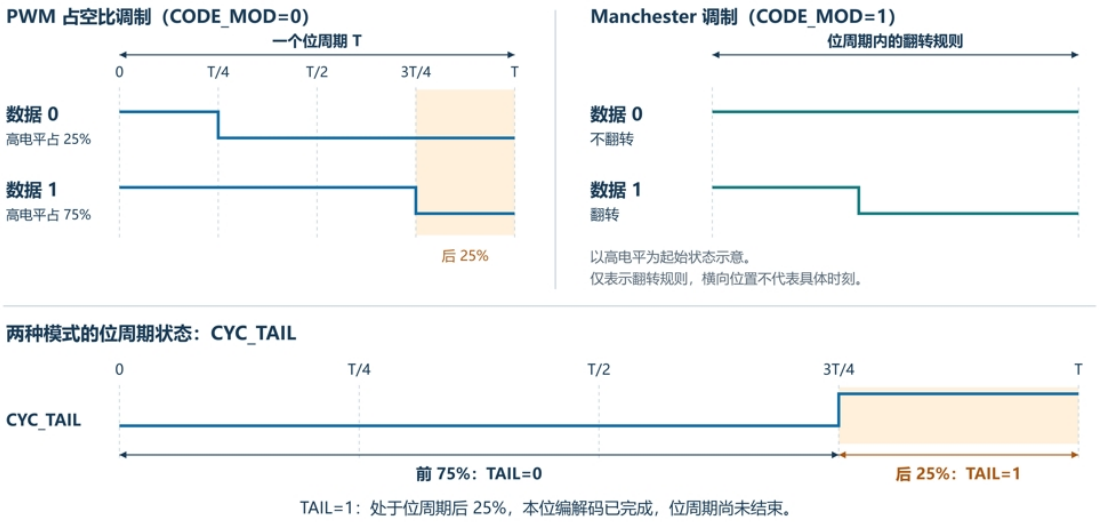


高电平：3×NOP + BC　低电平：3×NOP + JMP + 下一轮 BS

t 的单位为 PIOC 时钟周期；不表示内部写回相位或引脚传播延迟

位编解码用于逐位重复相同波形的协议：配置位宽与编码方式后，向 BIT_CYCLE 的 bit7 写入数据位，硬件即产生一个完整位周期，CPU 可同时处理下一位；PWM 模式下 0 占 25%、1 占 75%；Manchester 模式下 0 不翻转、1 在中点翻转；TAIL=1 表示已进入后 25%，本位编解码完成。写入 BIT_CYCLE 即启动位周期，无需额外的启动位；写该寄存器会清除 IO0 的边沿检测记录，若此时正在用 WAITB 等待边沿会受到影响。位宽字段写 0 时关闭编解码。

图 1-6 PIOC 两种位调制



WAITB 是 PIOC 中用于事件同步的指令。条件不成立时，程序计数器保持不变，反复执行该指令；条件满足后继续执行下一条。八种事件覆盖了主要同步需求：

等待主机写入命令（WB_DATA_MW_SR_1）；

等待主机读取结果（WB_DATA_SW_MR_0）；

等待位周期进入后 25%位编解码完成（WB_BIT_CYC_TAIL_1）；

等待 IO0 上升沿或下降沿（WB_PORT_IO_RISE/FALL）；

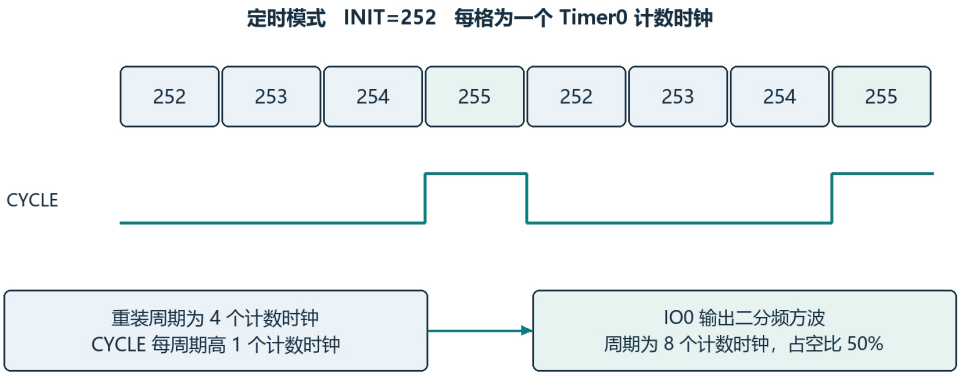
等待 IO0\IO1 输入与输出不同或恢复一致（WB_PORT_XOR0_1/0、WB_PORT_XOR1_1）。

两个语义细节影响其使用方式：边沿类事件在退出时会清除检测记录，因此连续两次等待上升沿不会因上次已检测到而立即返回；条件类事件若已成立则立即退出。PORT_XORx 表示输入与输出锁存值的异或结果。若 OUT0=1，XOR0=1 说明输入为低；若 OUT0=0，同样的 XOR0=1 却说明输入为高。因此 WAITB WB_PORT_XOR0_1 不能一律解释为“等低电平”，必须先明确 OUT 的参考值。

输入采样时刻还受 SB_PORT_IN_EDGE 选择：为 0 时在时钟周期中点采样，为 1 时取前一周期的边沿值。设计接收程序时，应先定位起始条件，再把采样点安排在有效窗口内，检测到边沿并不等于下一个指令就能采到位中心。

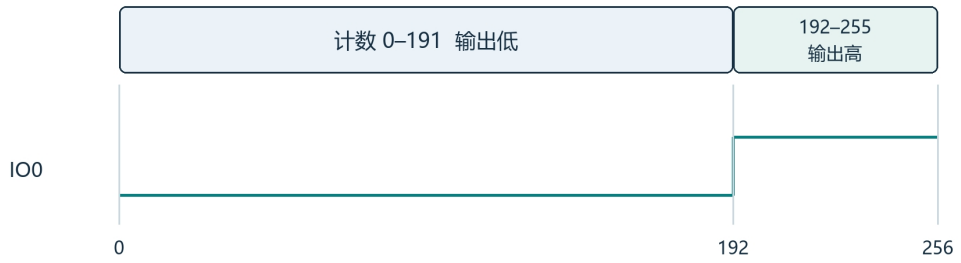
PIOC 提供一个 8 位定时器 0，TMR0_COUNT 保存当前值，TMR0_INIT 保存重装初值 I，TIMER_CTRL 选择分频、模式和输出。时钟由 SB_TMR0_FREQ 从系统主频分频得到，分频字段 000－111 依次对应 D=1024、256、64、16、8、4、2、1。定时器 0 可以工作于定时器模式或脉宽调制器模式。定时模式下，计数从 TMR0_INIT 计到 0xFF 后溢出并重装初值；PWM 模式的周期固定为 256 个定时器时钟，初值改为决定比较阈值。

图 1-7 定时器模式的计数



定时模式从 I 计到 255，再重装 I。重装周期为(256-I)×D/Fpioc；内部 CYCLE 在计数到 255 时为高，重装时回低。允许输出后，IO0 输出该周期信号的二分频方波，频率 Fpioc/[2×D×(256-I)]，占空比为 50%。

图 1-8 PWM 模式的计数

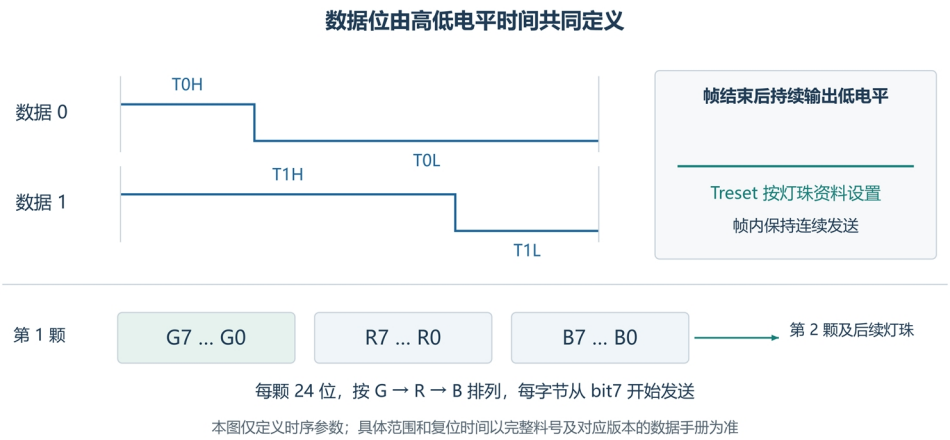


PWM 模式始终从 0 计到 255，再回到 0，周期为 256×D/Fpioc。INIT 在此模式中是比较阈值，不再改变计数周期；COUNT>I 时内部 CYCLE 为 1，在引脚由 PIOC 接管、方向为输出且定时输出允许时，该信号作用于 IO0。高电平占空比为(255－I)/256。

第2章 WS2812 单线驱灯示例

2.1 协议时序窗口

图 2-1 WS2812 位定义



判断时序是否合格，应分别核对 T0H、T0L、T1H、T1L 和帧结束复位低电平，0 与 1 的差别仅在于高电平占比。常见 24 位 WS2812B 每颗使用 3 字节，按 G、R、B 排列，每字节先发送 bit7。多颗级联时，顺序放入第 1 颗至最后一颗的数据；每颗截取属于自己的 24 位，其余数据经 DOUT 转发。以 00h、20h、00h 为例，表示绿色和蓝色分量为 0、红色分量为 20h。RGBW 等器件应另行确认字节数和色序。WS2812 其协议简单而时序要求严格：仅由 0 与 1 两种位构成，位宽容差±150ns，数据量可从小到大，且要求一次连续发送。本地例程 EXAM\PIOC\1_Wire 同时实现了短帧（SFR）与长帧（程序区载荷）两条路径，可用于说明同一协议的两种实现方式。

2.2 工程组成及初始化

表 2-1 示例工程主要文件

文件	阅读或修改重点
Asm/RGB1W.ASM	命令分派、SFR 与 RAM 取数、位时序、帧结束和结果返回
Asm/PIOC_INC.ASM	PIOC 侧寄存器、位号及专用指令参数
Asm/RGB1W.BAT	调用 WASM53B 汇编，并用 BIN_HEX 转换 C 数组
User/RGB1W.c 与 RGB1W.h	主机初始化、程序数组、缓冲区地址和发送接口
Ld/Link.ld	主机 RAM 范围及与 PIOC 复用 SRAM 的隔离

RGB1W_Init 依次使能 AFIO、GPIOC 和 PIOC 时钟，选择 PC18/PC19 或重映射后的 PC7，设置引脚复用；随后令 PIOC 保持复位且 CLK_GATE=0，将 PIOC_1W_CODE 复制到 PIOC_SRAM_BASE。该函数完成装载，实际发送接口再释放复位、开启 PIOC 时钟并提交任务。

主机链接脚本将普通 RAM 限制为从 0x20000000 开始的 16KB；PIOC 复用区从 0x20004000 开始，共 4KB。移植时保留这种隔离，避免主机的栈、堆或全局变量占用 PIOC 程序区。使用 PC18/PC19 时还需处理与调试接口的复用关系；仅使用重映射后的 IO0 时，应同步检查各发送函数中的 IO_EN 配置。

2.3 命令字与结果码

主机先准备数据和参数，最后写 CTRL_WR 提交命令。以下命令是例程约定的接口范围，不等于所有能进入汇编分支的数值都适合调用。RAM 路径将字节总数放在 DATA_REG0/1，将通道选择放在 DATA_REG2 bit0；SFR 路径则用 DATA_REG0～31 存放待发字节。

表 2-2 例程的命令字分配

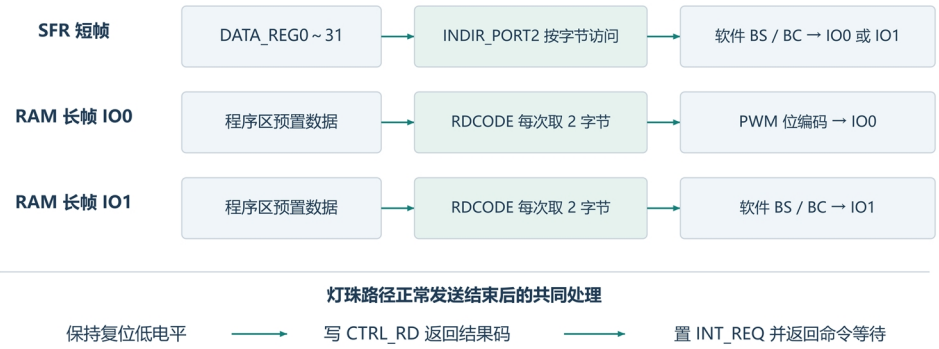
命令	含义	数据来源	通道
0x01 – 0x20	短帧，低 7 位为字节数（1 – 32）	DATA_REG0 起的 SFR	IO0
0x44 – 0x60	短帧，低 6 位为字节数	DATA_REG0 起的 SFR	IO1
0x88 – 0xFC	长帧，低 7 位为位宽 N	程序区载荷（RDCODE）	由 DATA_REG2 选择
0x41 – 0x43	DS18B20	—	IO0

IO1 短帧接口将长度与 0x40 合成命令。长度 1～3 对应 0x41～0x43，会先进入温度传感器分支，因此单颗 RGB 灯的 3 字节数据不能直接通过该原始 IO1 短帧命令发送。应重新分配冲突命令并同步修改 C 与 ASM，或采用已经修正且验证过的其他发送路径。用于完整 RGB 灯链时，字节数还应为 3 的整数倍。

任务完成时，PIOC 先写 CTRL_RD，再置 INT_REQ 并返回 WAIT_COMMAND。主机轮询等待接口通过 INT_REQ 判断完成，读取 CTRL_RD 取得结果；随后 PIOC 检查到 SW_MR=0，再清 INT_REQ。若改用中断方式，应由中断服务程序统一读取结果并通知应用层，避免轮询代码和中断重复读取同一个完成状态。

2.4 短帧与长帧路径

图 2-2 代码中三条路径



SFR 路径一次最多传递 32 字节，可容纳 10 颗完整 RGB 灯的数据。入口关闭位编码，设置对应 OUT 锁存值为低、配置输出方向，并检查输入是否为低。随后用 INDIR_ADDR 保存剩余字节数，用 MOVIA 将第二组间接地址 INDIR_ADDR2 指向 DATA_REG0。

```
        BS    SFR_PORT_IO,SB_PORT_OUT0   ; 上升沿
        CALL  RGB1W_DLY_0H               ; 高电平 292 ns
        BTSS  SFR_INDIR_PORT2,7          ; 数据位为 1 则跳过
        BC    SFR_PORT_IO,SB_PORT_OUT0   ; 数据 0：提前拉低
        CALL  RGB1W_DLY_HALF             ; 补齐至中点
        BC    SFR_PORT_IO,SB_PORT_OUT0   ; 数据 1：在此拉低
        CALL  RGB1W_DLY_1L               ; 低电平 292 ns
        ; … bit6～bit0 同样展开，最后一位使用 _B0 变体补偿指针推进
```

这段代码处理一个普通数据位。数据为 0 时，BTSS 不跳过第一次 BC，输出较早拉低；数据为 1 时跳过该 BC，保持高电平直到第二次 BC。DLY_HALF 用于补足中间时段，并不是把信号拉低到位周期中点。两条数据路径都要包含 BTSS 是否跳过及 CALL/RET 的开销。

图 2-3 软件位时序



0 码：高 14、低 42 周期　1 码：高 42、低 14 周期

横轴单位为 PIOC 周期，0 为 BS 置位生效的参考点；不画指令执行区间

DLY_0H 为 8 条 NOP 加 RET，共 10 周期；DLY_1L 为 11 周期；DLY_HALF 含嵌套调用及顺序执行到 DLY_1L 的路径，本体为 25 周期。计入调用和分支后，普通位的 T0H/T0L 为 14/42 周期，T1H/T1L 为 42/14 周期。48MHz 下分别约为 0.292/0.875us 和 0.875/0.292us，位周期约 1.167us。

bit7～bit1 的位检查不会自动推进数据指针；字节末尾通过 MOV SFR_INDIR_PORT2,A 触发 INDIR_ADDR2 自增，再递减字节计数。bit0 使用_B0 延时入口补偿取数和循环开销。修改时不能只调整普通位的 NOP，应同时核算字节末位、连续字节及最后一位后的收尾路径。

RAM 长帧的取数与位编码：

RAM 路径把数据预先放入 PIOC 程序区，借助 RDCODE 每次读取一个 16 位字。数据在 A 和 INDIR_ADDR 中分别得到低、高字节，因此先发送低字节，再发送高字节；每个字节内部仍按 bit7 到 bit0 发送。VAR_ADDR_L/H 另存下一次取数地址，VAR_SIZE_L/H 记录剩余字节数，DATA_EXCH 暂存当前字节。

IO0 采用硬件位编码：

IO0 分支将 BIT_CONFIG 设为 0x80，即 TX_EN=1、CODE_MOD=0。命令的低 7 位作为周期字段：48MHz 用 56，对应命令 0xB8；24MHz 用 28，对应命令 0x9C，例程标称位宽均约 1.167us。下列片段保留原始程序区读取地址。

```
        CLR   VAR_ADDR_L
        CLR   SFR_INDIR_ADDR
        MOVL  0X02                ; 原始字地址高位
        MOVA  VAR_ADDR_H
        RDCODE                    ; A=低字节，INDIR_ADDR=高字节
        MOVA  SFR_DATA_EXCH
        ANDL  0X80                ; 首字节 bit7
        IOR   SFR_INDIR_ADDR2,A    ; 合入周期字段
        NOP
        MOVA  SFR_BIT_CYCLE        ; 已使能发送，装入首位并启动
RGB1W_WORD_NEXT:
        WAITB WB_PORT_I0_RISE      ; 确认当前位已开始
        BP2F  BO_BIT_TX_O0,6       ; 预装下一位
```

这里的上升沿表示当前 PWM 编码位开始，不是当前位发送结束。TX_O0 为双缓冲结构，程序在当前位开始后装入下一位；当前位仍由编码器继续输出。WAITB 对上升沿检测记录进行等待，退出时清记录；写 BIT_CYCLE 同样会清 IO0 边沿记录，因此首位启动、记录清除和后续写位操作的顺序不能任意调整。

长度减 1 时，代码先执行 DEC VAR_SIZE_L，再执行 INC VAR_SIZE_L,A。若原低字节为 00h，减 1 后为 FFh，随后加 1 到 A 得到 00h 并置 Z，才对高字节减 1。第二条指令的结果写入 A，不会把已递减的 VAR_SIZE_L 改回原值。理解目的寄存器有助于避免在修改长度判断时破坏 16 位借位处理。

```
RGB1W_BIT_LAST:
        BP2F   BO_BIT_TX_O0,0
        WAITB  WB_PORT_I0_RISE        ; 最后一位开始
        BC     SFR_BIT_CYCLE,SB_BIT_TX_O0
        WAITB  WB_PORT_I0_FALL        ; 最后一位高电平结束
        CLR    SFR_BIT_CYCLE          ; 停止编码
        JMP    RGB1W_LOAD
```

末位装入后，代码先等该位上升沿，再清 TX_O0 并等待下降沿，随后停止编码并进入复位低电平阶段。BC BIT_CYCLE 也会清除旧的边沿记录，此处用来避免把此前的下降沿误认为末位结束。不能在刚写入末位后立即关闭编码器。

IO1 仍采用软件输出：

RGB1W_LONG_1 虽然同样从程序区取数，但 IO1 没有 IO0 的位编码输出功能，因此使用 BS/BC 和延时子程序生成波形。命令中的周期字段不能直接改变这一路软件脉宽。

2.5 主机调用与一次完整发送

初次接入可先使用 IO0 的 SFR 路径。完成系统时钟、接线、电平及首帧复位低电平准备后，以下调用向一颗灯发送低亮度红色数据；例程头文件和链接配置应已加入主机工程。

表 2-3 主机两种发送接口的操作顺序

阶段	SFR 路径
准备数据	在主机缓冲中排列 GRB 字节
复位与复制	复位后先启动 PIOC，再复制到 DATA_REG
提交任务	最后写长度与通道组成的命令
读取结果	等待 INT_REQ，再读 CTRL_RD
下一帧	处理结果并确认可再次提交

时序调整与重新生成固件：

若采用软件输出，可先确定各段的目标周期数，再修改汇编实现。修改 ASM 后，运行配套 RGB1W.BAT，检查 LST 的错误和警告、符号地址及代码末端；确认 BIN 转换生成的 RGB1W_inc.h 与 BIN 长度及字节内容一致，再更新 RGB1W.c 中的 PIOC_1W_CODE 数组。最后重新编译、下载主机工程，由初始化函数装载新的 PIOC 程序。只修改 ASM 或只重新编译主机工程，都不能保证运行的是最新 PIOC 机器码。

涉及 RAM 起点时，C 宏与两处 ASM 字地址应同步修改；涉及 24MHz 补丁时，应从当前 LST 确认补丁标签地址并换算为字节偏移。PIOC 程序地址以字为单位，C 指针以字节为单位，二者换算关系为“字节偏移=2×字地址”。

历史版本

日期	版本	变更内容
2026/11/02	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。